DOCUMENTATION · SECURITY

Threat model

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=THREAT_MODEL. This copy was made on 4 November 2026.

SECURITY.md tells you what to *do* - a checklist to run through before a deployment goes live. This page explains *why*: what QueryAPIGate is actually defending against, how each defense works under the hood, and - just as importantly - where the defenses stop and become the operator's job instead. Read it before deciding QueryAPIGate is (or isn't) an appropriate way to expose a database to something you don't fully trust.

What's actually at risk

QueryAPIGate sits between a caller and one or more real databases. Four things are worth protecting, in roughly descending order of damage if lost:

1. **The ability to execute arbitrary SQL/Mongo operations** against a connected database - the whole point of the service, and therefore its biggest liability if it reaches the wrong caller.
2. **Database credentials** - held in the store (`queryapigate.db`, or a PostgreSQL schema), needed to reach the databases at all.
3. **Query results and SQL text** - a saved query's SQL, and the rows it returns, can themselves be sensitive (customer data, schema details, business logic) independent of write access.
4. **API key secrets and the audit trail** - if either is compromised, every other control built on top of them (scoping, accountability) stops meaning anything.

Who might attack it, and from where

- **An unauthenticated network attacker** - anyone who can reach the HTTP port at all. The only thing standing between them and everything above is whether `QUERYAPIGATE_API_KEY` is set (see Open-access mode below) and whether TLS is actually terminated somewhere in front of the service.
- **A caller holding a scoped (non-admin) API key**, acting maliciously or - more realistically - a legitimate integration whose key has leaked. The threat here isn't "can they authenticate," it's "how far can that one key's specific grants actually reach."
- **An administrator** - the holder of the shared `QUERYAPIGATE_API_KEY` or of a named administrator's admin token - or anyone who steals one. What they can *change* depends on the role (below); but every role except auditor can run any SQL on every connection, so for the data each is equivalent to a database superuser scoped to whatever connections exist. This tier is not defended against by QueryAPIGate itself - see Non-goals.
- **Someone with filesystem or volume access** to `QUERYAPIGATE_HOME` - a compromised host, a misconfigured backup, a leaked volume snapshot. `queryapigate.db` is a single SQLite file holding connections, saved-query SQL, API key *hashes*, roles, execution history and the audit log - see Data at rest.

Security architecture, by control

Authentication: one shared secret, or many scoped ones

Every request carries (or omits) an `X-API-Key` header, checked once per request in `app.py`'s `before_request` hook, before anything else runs. Two kinds of key exist:

- **The admin key** (`QUERYAPIGATE_API_KEY`) is a single value living in the server's environment, compared with `hmac.compare_digest` (constant-time, so response timing can't be used to guess it character by character). It is never stored anywhere on disk - only in the process environment - and grants everything: every connection, every saved query, admin-only routes (managing connections, keys, roles).
- **Admin tokens** (`qagadm_...`) belong to named administrators (`admins.py` , ADR 0003): generated by the server, shown once, stored only as a SHA-256 hash and looked up by it, like scoped keys - but in their own table, and they authorize management by role rather than data access by grant. Each can carry an expiry; deactivating an administrator stops all of theirs at once.
- **Scoped keys** (`POST /api/v1/api-keys` , owners and admins to create) are generated by the server and shown exactly once - their secret is never stored, only a SHA-256 hash, in `queryapigate.db` 's `api_keys` table. A lost scoped key cannot be recovered, only revoked and replaced, the same property a password reset flow gives you and a plaintext-stored secret never could. A key's secret can be **reissued** (ADR 0009, `POST /api/v1/api-keys/{name}/rotate` , `access.write`): the new one is shown once, and the old one's hash keeps authenticating for a grace period the administrator chooses - at most a week, 0 for a leaked secret - under the same `active` , expiry, `allowed_ips` and rate limit. Only one old secret at a time: a second rotation during the grace is refused (`key_rotation_pending`), and `DELETE .../previous` ends the grace early. When the old secret was last used is shown, so it isn't ended while still in use - or left running after it should have been.
- **Signed-in users** (`Authorization: Bearer` , optional - `jwtauth.py`) carry a JWT from the deployer's own login or identity provider instead of a key. The signature is verified against a shared secret or the provider's JWKS; `exp` is required; `iss` / `aud` are enforced (mandatory with a JWKS URL, so a token minted for another application by the same provider is refused); the accepted algorithms are fixed by configuration and can't mix HMAC with public-key families (algorithm confusion), and `none` is never accepted. Grants come from a role, read live. A saved-query parameter declared `from_claim` is bound from the verified token, never from the request, which closes the "change `user_id` in the URL" class of bug (insecure direct object reference) for queries written that way. "Written that way" is the gap: a query granted to the same role without such a parameter returns every user's rows. A role with `require_claim_binding` closes it - its users run only queries that bind their token, by name or collection, never a connection or ad-hoc SQL - and is the setting for a role whose users are customers. Residual risk: a JWT can't be revoked before it expires.

Open-access mode: a deliberate default worth understanding

If `QUERYAPIGATE_API_KEY` is never set, no scoped keys or administrators exist, and JWT and MCP OAuth are off, QueryAPIGate treats every caller as an implicit, unrestricted admin - but **answers local (loopback) clients only**. A request whose real socket peer isn't a loopback address is refused `403 open_access_remote` , pointing the operator at `QUERYAPIGATE_API_KEY` . The peer is the actual TCP connection, read before `ProxyFix` , so a forged `X-Forwarded-For` can't make a remote caller look local; an address that can't be parsed (or none, as over a Unix socket) counts as remote. The public endpoints (`/health` , `/` , `/docs` , `/openapi.json` , the Console shell) stay reachable, as they are even when a key is set.

So the zero-config local trial still works, and the "deployed open on a network by accident" footgun is closed: exposing the port no longer exposes the data. This is aimed at a local trial or a fully trusted internal network where standing up a key first would just be friction. It is still **not** how to run a server other parties use: the moment more than loopback should reach it, setting `QUERYAPIGATE_API_KEY` (or configuring administrators, JWT or MCP OAuth) stops being optional - SECURITY.md's first checklist item. Behind a reverse proxy or in a container, the peer is the proxy or the pod network, not loopback, so open-access mode is refused there too: set a key.

The MCP server (`queryapigate mcp`) applies the same rule to every request to `/mcp` , before a session starts, and more strictly: a caller is local only if the connection's own address is loopback and so is the client a trusted proxy names, if one does. So a forged `X-Forwarded-For` , even with `FORWARDED_ALLOW_IPS` wide open, doesn't make a remote client local, and neither does a proxy on the same host passing a remote client on. *Fixed 2026-10-30:* until then it didn't apply it at all. With no key, administrator, JWT or OAuth configured, any client that could reach its port was the unrestricted caller - every saved query, read-only SQL on every connection, an agent's writes. Its default address is loopback, but the Docker image needs `--host 0.0.0.0` , so a container started without a key served the network. Found while drafting the MCP Registry entry, which is exactly that command.

Authorization: what a scoped key can actually do

Three grant fields say what a key reaches, and they **add up** - a saved query is reachable if `connections` , `collections` or `queries` reaches it. The rest **narrow** what that reach may do, and none of them widens what another restricts:

GRANT	NARROWS
`connections`	Which database connections ad-hoc SQL can target (`"*"` = all).
`collections`	Which saved queries are reachable, by current collection membership - re-evaluated live, no wildcard.
`queries`	Specific saved-query names reachable regardless of connection grants - the only way to give an external client curated access with *no* ad-hoc SQL capability at all.
`allow_writes`	Whether write statements are permitted at all, on top of the server-wide `QUERYAPIGATE_ALLOW_WRITES` ceiling (a key can never grant itself more than the server allows).
`allowed_write_ops`	Which specific write verbs (insert/update/delete/...) if writes are allowed.
`allowed_tables`	Which tables a statement may touch - enforced via a real SQL parser (`sqlglot`), not a regex; unsupported for H2/JDBC connections, which are refused outright for a table-restricted key rather than silently under-checked.
`rate_limit`	A per-key request budget, additive with the server-wide IP-based limit, never a way around it.
`expires_at`	A day after which the key stops authenticating at all, checked live on every request.
`allowed_ips`	Which source IPs/CIDRs the key may be used from at all - fails closed if the client IP can't be resolved.

Roles (`POST /api/v1/roles`) are templates, not live policy: creating a key `from` a role copies that role's grants onto the key once, at creation time. Editing or deleting a role afterward has no retroactive effect on keys already created from it - a role is a way to stamp out several similarly-scoped keys quickly, not a group membership the system re-checks later.

Administration: named administrators and roles

The Management API (`/api/v1`) is for administrators only. Every operation needs one capability, and each of four fixed roles holds a fixed set (`adminroles.py`), checked before any route runs; a test fails if an operation has no entry, so a new route can't ship unguarded. A scoped key or signed-in app user gets `403 admin_only` ; an administrator whose role lacks the capability, `403 role_forbidden` .

ROLE	CAN CHANGE	DATA PLANE
owner	everything, including administrators	any SQL, every connection
admin	connections, API keys and roles, queries, cache - not administrators	any SQL, every connection
developer	saved queries and collections	any SQL, every connection
auditor	nothing - reads configuration, the audit log and history	none

What this buys, and what it doesn't:

- **A stolen developer token can't mint keys, change connections or create administrators** - but it can read every row on every connection. Roles bound *management* blast radius; for data, the database account behind each connection is still the backstop. Give someone a scoped API key instead when they need less data reach.
- **Who can call what is an owner's or admin's decision.** Moving a query into a collection, or merging collections, in a way that gives an API key or role new reach needs `access.write`, even for a role that may otherwise organise queries. A developer still decides what saved queries *return* - a new version of a query a partner calls, a new query in a partner's collection - which the audit log and drafts make reviewable, not preventable.
- **The shared key stays an owner** (break-glass). Once an active owner exists, each use is logged, audited (`break_glass_used`) and alerted, so a leaked shared key in use shows up; owners can then remove it.
- **Lock-out and reopen-by-accident are refused**: while any administrator exists, authentication stays required even with no shared key, and the last active owner can't be demoted, deactivated or removed without the shared key set. `queryapigate admins token NAME`, run on the server, always recovers - anyone with shell access to the store is already beyond this model.
- **Self-service is narrow**: an administrator may issue and revoke only their own tokens; anyone else's need an owner. An administrator and an API key can't share a name, so actors in logs and history are unambiguous.
- **Residual risks**: tokens are bearer secrets with day-granularity expiry; there is no MFA or SSO yet (BACKLOG #84 Phase 2), so a token is only as safe as where it's kept.

Console sign-in through SSO (ADR 0007)

With `QUERYAPIGATE_SSO_*` set, administrators sign in through the organisation's identity provider; the server runs the authorization code flow with PKCE as a confidential client, so the browser never holds a token.

- **The callback can't be forged or replayed.** A sign-in is stored server-side with its state, nonce and PKCE verifier, used once (a racing duplicate is refused), within ten minutes, and only in the browser that started it - so someone can't send you a link that signs you in as them. The ID token's signature (the key's own algorithm), issuer, audience (`azp` too, with several audiences), nonce and expiry are all checked. After sign-in the browser only ever goes to a Console page - never an address from the request.
- **The session can't be used from elsewhere.** Its cookie is `__Host-`, `HttpOnly`, `Secure`, `SameSite=Strict`, stored only as a hash; and it authenticates a request only together with the session's CSRF token, which only a page on this origin can read. A link, form or image on another site can't act as an administrator, whatever the method. `/auth/*` never answers cross-origin. Signing in again ends the session the browser had.
- **People are bound by the provider's `sub`**, never by an email. Linking an administrator who had tokens matches a *verified* email, once, and their role becomes their groups' - a pending owner account gives no one owner rights. Started sign-ins are capped, so starting them can't grow the store without bound.

- **What remains with the provider:** who is in the groups mapped to roles - **whoever controls the owners' group controls QueryAPIGate** - and whether `email_verified` can be trusted for linking. Someone removed there keeps a session already open until it expires (`QUERYAPIGATE_SSO_SESSION_HOURS`, default 8); switching them off in QueryAPIGate ends it at once. `QUERYAPIGATE_SSO_ONLY` takes personal tokens from people who sign in through SSO, so they can't route around the provider's MFA; the break-glass key stays, audited and alerted, for when the provider is down.

Reviewed for 0.30 (ADR 0017). Looked at: PKCE with S256, the state bound to the browser and taken once, the nonce, `azp`, the ID token's algorithm from the provider's key and never from the token; `return_to` only ever a Console page; linking only to an administrator an owner invited for that verified email; the role from the provider's groups every sign-in; the session cookie (`__Host-`, `SameSite=Strict`) and its CSRF token, reads included; signing in again ending the session the browser had. **Found:** a refused sign-in sent the Console its reason in the address - `?sso_error=...&sso_message=...` - and the Console showed `sso_message` as it was written, so anyone could send an administrator a Console link that said what they liked in the Console's own error banner ("call this number"). Now only the code travels, and the Console says what each one means in its own words; the reason, with whatever the provider said, is in the audit log.

Workspaces: one team's reach, not another's (ADR 0023, experimental)

Workspaces (ADR 0023) let several teams administer one server: each owns its connections, saved queries, keys, roles, destinations and exports. The boundary they draw is **between teams who share a server**, not between a hosted service's tenants. What it holds against, and how:

- **Every lookup is in one workspace.** A request's workspace is settled once (`app.settle_workspace()`, kept on the request, never on a thread a later request reuses) and every statement on a workspace's table filters on it - names are unique per workspace in the store's own keys, so there is nothing for a forgotten filter to match by name alone across workspaces. Work that outlives the request - a stream's rows, queued history, events and usage counts
  - carries the workspace it was started in. `tests/test_workspace_isolation.py` builds two workspaces with the same names for everything and checks each route reaches only its own: a lookup that forgot its workspace shows up there as the other workspace's data.
- **A key can't leave its workspace**, prefix or not: its secret says which workspace it is in. Naming another, or one that doesn't exist, is the same `404` - a caller can't use the error to list workspaces.
- **A signed-in user can't leave their workspace either.** A federated (JWT) user's workspace comes from the token - the `QUERYAPIGATE_JWT_WORKSPACE_CLAIM` claim, or the default workspace when that isn't configured - never from the request path. The role is looked up in that workspace, and a path naming a different one is the same `404` as for an API key. So a user signed in for one team can't reach another team's data by changing the `/w/<workspace>/` prefix, even where both workspaces happen to have a role of the same name. (Before this was enforced, the role was resolved in whatever workspace the path named - the finding the 0.39 boundary review turned up and closed.)
- **A role from a membership is bounded twice**: its reach is its workspace's connections only, and it never holds what only a server-wide role does (`adminroles.SERVER_ONLY`): administrators, settings, re-keying, erasing a person, creating workspaces, `/metrics` and the instance list. A member naming a workspace they aren't in gets `404`.
- **What happened is per workspace**: audit entries carry their workspace inside the hash chain, so rewriting whose an entry was breaks the chain; a member reads only their workspace's. Events reach only their workspace's subscribers; the response cache, and its management, are per workspace.

What it doesn't protect against - stated in the documentation's first paragraph about it:

- **The people who run the server.** Server-wide owners and admins see and change every workspace, by design.

- **Shared infrastructure.** One process, one store, one encryption key for every workspace's connection passwords. A bug outside the lookups above - in the process, or in a driver - is a bug for every workspace. No per-workspace resource limits yet: one workspace can exhaust the shared connection pool, event-stream slots or database load for the others (each key's own rate limits and quotas still apply).
- **The JVM, for a workspace owner.** Creating a connection is a workspace owner's to do (`connections.write` isn't server-only), and an experimental H2 or JDBC connection is opened read-write on one shared JVM with only the SQL guard as its lock - so the H2/JDBC process-level risk above now reaches any workspace's owner, where before it took a server administrator. Leave H2/JDBC off (they are experimental) where workspace owners aren't fully trusted.
- **The databases themselves.** Two workspaces can each have a connection to the same database host; what a workspace's connection can read is its database account's grants, as it always was.
- **Folders.** A local-folder destination is allowed only in `default`: `QUERYAPIGATE_EXPORT_DIRS` is the server's, and two workspaces writing under it could reach each other's files.

Workspaces are experimental until a security review of this boundary, as `experimental.GRADUATION` requires of anything that guards access.

Column masking: which values leave

A `mask` grant on a key, role or export (ADR 0005) hashes, masks, nulls or drops the values of named source columns in everything that caller receives. The design choice that matters: **result columns are traced to their sources through the SQL** (sqlglot lineage, by position, with the connection's schema for `SELECT *`), not matched by name - so `SELECT email AS e`, `lower(email)`, a CTE or a `UNION` is still masked - and **anything that can't be traced is refused before it runs**, never served unmasked. Probing during the build found, and the tests now hold against, the ways tracing could be led astray: DuckDB's `COLUMNS('regex')` (columns picked by pattern), a table alias that renames columns (`customer AS c(a, e)`), `SELECT *` over a `NATURAL` / `USING` join (fewer columns than parsed, which would shift every mask), whole-row references (`row_to_json(c)`), and a `VALUES` row holding a subquery. The database's error text, which can quote a value, is never returned to a caller with rules. The cache is keyed per set of rules, and run history records what was masked.

What remains:

- **Inference.** A caller that writes SQL can filter and group on a masked column (`WHERE email LIKE 'a%'`, `GROUP BY email`) - masking hides values in results, not answers about them. For callers near sensitive data, especially AI agents, grant saved queries only, or `allowed_tables` on a view that leaves the column out.
- **Hashes show equality.** `hash` is keyed (HMAC; the key never leaves the server), so a guessed value can't be hashed and compared, but equal values give equal codes - by design, for grouping and joining.
- **Tracing is only as good as the parser.** A construct sqlglot reads wrongly, rather than not at all, could trace a column to the wrong source. The refusal rules are deliberately broad, and the feature is experimental for that reason; database-level column privileges remain the backstop for a column that must never leave.

Query execution: what actually stops a bad statement

This is the part worth being the most precise about, because it's also the part with the sharpest edges.

- **Bound `:name` parameters are real parameterization.** `sqltools.bind_parameters()` rewrites `:name` into the target driver's native paramstyle and hands values to the driver separately from the SQL text - the same mechanism that makes parameterized queries safe in any language. Values never become part of the SQL string, so this path has no SQL-injection surface by construction.

- **The read-only / single-statement guard is a regex-based classifier, not a SQL parser.** It strips string literals and comments (dialect-aware - MySQL/ClickHouse honor backslash escapes inside string literals by default, PostgreSQL/SQLite/H2 do not, and the guard accounts for the difference), checks for a bare `;` outside what's left to block multi-statement smuggling, and classifies the leading keyword against an allow-list when writes aren't permitted - then looks behind a read-only one for writes it can hide: a writable CTE (`WITH d AS (DELETE ... RETURNING *) SELECT ...`), the statement `EXPLAIN ANALYZE` runs, and `SELECT ... INTO`. The same rule covers a key's `allowed_write_ops`. PostgreSQL, MySQL, ClickHouse and SQLite connections also run read-only queries read-only in the database (a read-only transaction on PostgreSQL and MySQL, a read-only setting or file mode on ClickHouse and SQLite), so the database refuses a write even if the guard missed it; **DuckDB, H2 and JDBC connections are opened read-write, so there the guard is the only lock** - the case where 0.16 closed a real gap (`EXPLAIN ANALYZE DELETE` ran on DuckDB with writes off). **This is explicitly documented as defence in depth, not a substitute for least-privileged database grants** - a regex can approximate a SQL parser's boundaries but can never fully replace one, and the project has already found and fixed a real gap of exactly this kind (see below).
- **`allowed_tables` uses a real parser** (`sqlglot`) specifically because table extraction has to be correct, not just approximately right - a key restricted to certain tables is refused entirely rather than let through on a best-effort guess when the dialect can't be parsed reliably (H2/JDBC).
- **`allowed_tables` is matched schema-qualified** (from 0.43). Before, every dialect but Trino matched bare names, so a grant of `customer` read `billing.customer` too, on a connection whose user could see both. Now a reference's and a grant's missing schema is the connection's default (`engine._table_defaults`): MySQL's or ClickHouse's database, PostgreSQL's `public` or `options.schema`, SQLite's and DuckDB's `main`. A catalog a reference names stays part of its name (fail closed). PostgreSQL resolves unqualified names through `search_path`, so a table-restricted call pins it to that schema, inside the call's own transaction. Names are compared lower-cased, so two tables differing only in case (MySQL with `lower_case_table_names=0`, quoted PostgreSQL names) share a grant.
- **Statements the check can't see into are refused for a table-restricted key:** `SHOW`, `SET`, `USE`, `ATTACH`, `PRAGMA`, `CALL`, `PREPARE`/`EXECUTE`, `DECLARE`/`FETCH`, and anything sqlglot only keeps as text (`tableguard._statements`). Each names its tables somewhere the check doesn't look, or none at all: before 0.43, `SHOW CREATE TABLE secret` showed a forbidden table's DDL, `PREPARE` then `EXECUTE` read its rows, and DuckDB's `SET search_path` rerouted later calls on a pooled connection. `EXPLAIN` is checked by the statement it explains.
- **Nothing a PostgreSQL query sets outlives it** (ADR 0027). Calls run in autocommit, where a session setting a query changes (`SELECT set_config(...)`) is committed with it, so each message begins `RESET ALL`. Read-only, the time limit and the pinned `search_path` are set `LOCAL`, inside the message's transaction.
- **DuckDB file access is enforced by DuckDB itself.** A DuckDB connection reads only its `allowed_paths` - none by default - because each connection's configuration is locked as it opens (`enable_external_access = false`, the allowed paths, `lock_configuration`): no SQL can reach another file, URL or extension, or change the setting back. Object-storage prefixes are a second lock behind the credentials' own permissions, since an S3-compatible server could resolve `..` in a key; web addresses must name files for the same reason.
- **A local-folder destination is off unless the operator turns it on** (`QUERYAPIGATE_EXPORT_DIRS`), can only be a folder inside one listed there, and is never the server's home - so an administrator can't write a file over the store, or anywhere else the process can reach, through an export. Object-storage destinations need no such list: the credentials bound them. Checked when the destination is saved and again before each write.
- **A connection can't be pointed at the server's own metadata store** (`store_not_allowed`): neither a SQLite/DuckDB/H2 `database` of `queryapigate.db` nor a DuckDB `allowed_paths` folder containing it - which would otherwise hand an administrator the key and token hashes. A PostgreSQL store is kept apart by its own credentials.
- **A `${VAR}` reference can't name one of the server's own `QUERYAPIGATE_*` settings** in a connection or destination - otherwise an administrator could carry the admin key or secret key out in a password sent to a host of theirs. **Nor a credential the server itself may run with** - `AWS_*`, `AZURE_*`, `ARM_*`, `GOOGLE_*`, `GCP_*`, `VAULT_*`, `KUBERNETES_*` -

which would reach beyond every allow-list (found in the review for 0.30: `${AWS_SECRET_ACCESS_KEY}` was accepted, and with it the server's whole AWS identity, not just the secret prefixes `QUERYAPIGATE_AWS_SECRET_PREFIXES` opens). Any other variable is reachable until `QUERYAPIGATE_SECRET_ENV_VARS` lists the ones that may be: **set it**, or a credential under a name no rule knows (`DB_ADMIN_PW`) is an administrator's to send. The server warns at startup while references are in use without it.

- **Exports write only under their destination's prefix** (ADR 0004). A destination is a bucket prefix or folder and write credentials, stored like a connection password. Each run writes through a fresh DuckDB connection locked the same way - `allowed_paths` is exactly the prefix plus the one temporary file the rows were staged in - and object paths are checked before DuckDB sees them: a parameter value in a path may only be a plain name, `..` and absolute paths are refused. So a path template, parameter or file name can't reach another partner's prefix. The residual risks: credentials that can write more than the prefix (scope them, as for reading), and *what* is delivered - defining an export decides who receives which data, so only owners and admins may (`exports.write`), and every change to one is audited; a developer may only run one already defined.
- **Older `{name}` text placeholders are string substitution**, not parameterization - restricted to a narrow safe-character pattern (digits and plain text only) specifically because they're substituted directly into the SQL text. Prefer `:name` bound parameters; `{name}` exists for cases (identifiers, table/column names) that can't be a bind parameter in any SQL dialect.

A real bypass was found and fixed, and is worth citing as the honest version of "defence in depth": the fuzz test suite (`tests/test_sql_guard_fuzz.py`) documents that MySQL/ClickHouse's backslash-escape handling inside string literals let a crafted literal hide a `;`-separated second statement from an earlier, dialect-naive version of the guard. It happened not to be exploitable in practice at the time (no driver call ever asked for a smuggled statement's results, and read-only sessions blocked it anyway), but the project's own account of it calls it "a real failure of an explicitly documented guarantee" that "directly undermined auditability" - which is why the guard is now dialect-aware and exercised by a fuzz test on every change, not just trusted to keep working.

MongoDB: a find() and nothing else

A `mongo` connection has no SQL guard to rely on; what keeps it read-only and inside its grants is that only one operation exists. Reviewed for its graduation to tier 1 in 0.37 (ADR 0022):

- **Only `find()` is ever sent** - no aggregation pipeline, no write, no command a caller can name. There is no `allow_writes` to get wrong.
- **No server-side JavaScript.** `$where`, `$function` and `$accumulator` are refused in the filter, the projection and the sort (`mongotools.py`), on every request and when a saved query is saved. Until 0.34.2 only the filter was checked, and MongoDB 4.4+ evaluates `$function` in a projection: any key with a MongoDB connection could run JavaScript on the database server.
- **A parameter is a value, never operators.** A `:name` placeholder takes a string, number, boolean, null or a list of those - `{"$ne": null}` is refused (`param_invalid`), not run as a filter matching the whole collection. Until 0.34.2 an untyped placeholder took any JSON.
- **`allowed_tables` is the collections a key may read**, checked before the server is asked anything - in paged runs, streams and exports. Until 0.34.1 it wasn't checked on MongoDB at all. Collection names are compared case-insensitively, as table names are, so `orders` in a grant also reads a collection named `Orders`: a limit, documented with the type.
- **The time limit is the cursor's `maxTimeMS`**, over the whole cursor - a stream included - and the caller's concurrency slot is held until the last document is read or the client leaves, when the cursor is closed on the server.
- **Column masking is refused** on MongoDB (`mask_unresolvable`) rather than applied to fields it can't trace.

- **The refusals are properties under fuzzing**: `fuzz/fuzz_mongo.py` (ClusterFuzzLite, from 0.35.1) checks that no server-side JavaScript survives into what MongoDB is sent and no parameter becomes an operator; the CI integration job sends them to a real MongoDB (7.0 while the CI runners' kernel stops 8.0 starting; see ci.yml).

Trino: a federation engine behind one grant (ADR 0026, experimental)

A `trino` connection reaches every catalog the Trino cluster federates - Hive, Iceberg, Kafka, other databases - through one QueryAPIGate connection. That reach is the point, and the risk: the controls other SQL connections have within one database must hold *across catalogs*. Reviewed for its introduction (experimental, ADR 0026):

- **`allowed_tables` is matched fully qualified** (`catalog.schema.table`), not by bare name. A reference's missing parts are filled from the connection's default catalog (its `database` field) and schema, and a grant is filled the same way (`tableguard.qualify_grant`). So a grant of `orders` reaches only the default catalog+schema's `orders` - never `orders` in another catalog or schema - and `hive.sales.orders` reaches exactly that table. This is the federation-specific fix: bare-name matching would have let one grant read the same table name in every catalog the connection can see. The schema browser applies the same qualified grants (`schema._only_allowed`).
- **A data-reading table function is refused.** `tableguard` passes only pure value generators; a Trino `TABLE(...)` polymorphic function that reads data - `system.query(...)` (a pass-through that would run arbitrary SQL on a backend), `table_changes(...)` - names no base table the allow-list can account for, so a table-restricted key is refused (`table_not_allowed`), as `read_parquet()` / `url()` are on other dialects.
- **Read-only rests on the SQL guard alone** (Trino has no read-only session mode), as for DuckDB, H2 and JDBC. A non-read-only statement is refused with `read_only` before it runs, including a write hidden behind a read-only first keyword - a writable CTE, `EXPLAIN ANALYZE` (`sqltools.write_operations`). Trino's write surface is covered: INSERT/UPDATE/DELETE/MERGE, `CREATE TABLE AS`, `CALL`, and `ANALYZE` (which collects statistics, treated as a write).
- **Masking is traced with sqlglot's `trino` dialect** and fails closed: a query masking can't follow is refused (`mask_unresolvable`), never served unmasked - as on every SQL connection.
- **Parameters are bound, never interpolated** (qmark); only a single statement runs; the time limit **cancels** the running query on the cluster (`cursor.cancel()` on a timer, plus a `query_max_run_time` session backstop), releasing its resources rather than merely abandoning it.
- **The Trino user is the outer boundary.** A key granted the whole connection (not a table list) gets read-only ad-hoc `execute_sql`, and can reach every catalog and schema the *Trino user* can; QueryAPIGate narrows within that, never beyond it. So a Trino connection must use a **read-only, catalog/schema-scoped Trino user** - Trino's own access control is the floor, QueryAPIGate's grants the ceiling. This is the one operational control an operator must get right, and it is stated with the type.
- **Tested against a real Trino in CI** (`tests/test_integration.py::TrinoTests`): the read-only refusal, the time limit, masking, and the qualified `allowed_tables` (a bare grant refusing another catalog) each run against a live Trino, not only in unit tests.

What remains before tier 1 is a release of unchanged shape and real-deployment use; agent writes (ADR 0008) are not offered on Trino, and the schema browser not showing keys or DDL is a capability gap, not a security one.

The Rust data plane (ADR 0028, a prototype)

A second process, `dataplane/`, can listen on the public port in front of the Python server. It serves `/q/<name>` reads itself from *plans* Python gives it, and forwards everything else unchanged (ADR 0028). It's in the Docker image, and the Helm chart's `dataplane` block runs it as a second container in each API server pod (ADR 0030), but it runs only when started: nothing changes for a deployment that doesn't. It isn't in the PyPI package. Where it does run:

- **It makes no security decision of its own.** The SQL guard, the table allow-list, masking and every refusal stay in Python. A plan is Python's verdict for one key, one saved query version and one connection, made only on a call that succeeded, and only for a scoped API key in the `default` workspace with no rate limit, quota, concurrency, time or row limit of its own. Administrators, signed-in users (JWT, SSO, MCP OAuth), other workspaces, writes, streams, other formats and every dialect but PostgreSQL and MySQL are always forwarded. With a server-wide rate limit or OpenTelemetry tracing configured, it forwards every call.
- **Behind trusted proxies, it finds the client as Python does.** With `QUERYAPIGATE_TRUST_PROXY` set, the address a key's `allowed_ips` is checked against comes from `X-Forwarded-For`, as `ProxyFix` takes it, so this is an access decision made in two places. The data plane serves only a call whose forwarding headers read one way: one line each, no quotes, no empty item, an address in its own spelling, a scheme of `http` or `https`, and a host werkzeug can encode. Anything else is forwarded for Python to read. The differential test's proxied run sends both every shape of those headers, against IP-restricted keys, and requires the same answers (ADR 0030, section 5).
- **Behind it, gunicorn trusts no client's forwarding headers.** Every request reaches gunicorn over loopback, and gunicorn trusts loopback by default to set the scheme (`X-Forwarded-Proto`, `-Ssl`, `-Protocol`) and `SCRIPT_NAME` from headers. A client's own were passed on, so it could make Python send HSTS over plain HTTP, turn a call into a 400, or move part of a path out of what Flask routes, past a proxy's rules for that path. With the secret set, `gunicorn_conf.py` trusts no peer's (`forwarded_allow_ips` empty, unless `FORWARDED_ALLOW_IPS` says otherwise), and Python reads the forwarding headers only through `ProxyFix`, as it does without the data plane. Found and fixed shipping it, 2026-10-30.
- **The two come from one release.** The protocol between them is internal and unversioned, so the data plane sends its version with the secret (`X-QAG-Dataplane-Version`), and Python gives no plan and no run store lease to another. A mismatched pair forwards every call. In the image both come from one build, and the crate's version is the release's, checked by a test.
- **A plan is checked against the store on every call.** It names digests of the key, query and connection rows it was computed from. The data plane reads those rows for each call, with Python's own SELECTs, and serves only when they digest the same. A revoked, expired or re-scoped key, an edited query or a changed connection doesn't match, and the call goes to Python. With `QUERYAPIGATE_METADATA_CACHE_TTL` set, both processes reuse those rows for the same number of seconds, so a change takes effect within that time, as it does on Python alone. A masked plan expires with the schema its trace was made from. Plans live in the data plane's memory only.
- **One shared secret ties the two processes together.** `QUERYAPIGATE_DATAPLANE_SECRET`, at least 16 characters, goes in `X-QAG-Dataplane` on every request the data plane forwards; Python compares it in constant time. The data plane removes `X-QAG-Dataplane`, `X-QAG-Peer` and `X-QAG-Plan` from anything a client sends, and Python's outermost WSGI layer removes the first two again before anything else reads or logs a request. Without the secret, Python gives no plan, ignores `X-QAG-Peer`, and answers 404 at `/internal/dataplane/runs`.
- **The client's address is the client's.** With the secret, `X-QAG-Peer` carries the client's own socket address, and Python takes it as the real peer before `ProxyFix`. So open-access mode's loopback rule, a key's `allowed_ips` and `QUERYAPIGATE_TRUST_PROXY` judge the client, not the data plane's loopback connection. A peer that isn't an IP address counts as remote.
- **Parameter values go into the SQL as literals,** as psycopg2 and mysql-connector do, so the quoting is the injection boundary. PostgreSQL strings are sent as pure-ASCII `E'...'` with `\u` escapes, and MySQL strings as `_utf8mb4` `X'...'`, so neither depends on a session setting a query could change (`standard_conforming_strings`, `sql_mode`). Each PostgreSQL message ends with `RESET ALL`. The differential test sends hostile strings for exactly this.
- **Its runs are recorded by Python.** The data plane reports each served run to `/internal/dataplane/runs`, with the secret, and Python writes its history, event and metrics. Python trusts those entries: whoever holds the secret can write run history. Runs the data plane can't queue, and reports Python doesn't take, are counted as dropped in

`/metrics`, never lost silently. On SIGTERM it sends what it still holds before it exits, for up to 10 seconds, as Python's own history writer flushes at exit; a hard kill loses up to one flush interval of runs, as Python's does.

- **Resource use before the key is checked is bounded.** The data plane reads a request's body itself only when it has a Content-Length of at most 64 KiB; anything larger, or chunked, streams through to Python unread. A connection whose database can't be reached is left to Python for 30 seconds, rather than each call waiting on a connect.
- **What the process holds:** the store's credentials (`QUERYAPIGATE_DATABASE_URL`, or the SQLite file) for reading, each connection's credentials as the store keeps them, and the shared secret. A compromise of the data plane is a compromise of what those reach, as for the Python process. Run it as the same user, with the same secrets, and never expose Python's own port once the data plane is in front of it. The image and the chart put Python on the pod's loopback (`QUERYAPIGATE_BIND=127.0.0.1:5001`), reachable from its own pod only.

Reviewed for 0.44. The gate is the differential test (`dataplane/tests/test_differential.py`): random calls sent to both processes, comparing whole responses and the history rows they leave, with expired and rotated keys, hostile parameters, and grants changed, a key revoked and a query given a new version while calls run. Every run of it was clean, over 13,500 calls on PostgreSQL and MySQL. The data plane's Rust dependencies are pinned in `dataplane/Cargo.lock` and updated by Dependabot.

Reviewed for shipping it (ADR 0030). The differential test also runs behind two trusted proxies with HSTS on: 9,000 more calls over three seeds, every one the same. CI audits the crates against RustSec (`cargo audit`) and fails on a known vulnerability, and the image's binary is built with `cargo auditable`, so the image's SBOM and scanners see its crates. Its cryptography (ring, RustCrypto) isn't a FIPS-validated module: a deployment that needs FIPS 140 leaves the data plane off.

The ClickHouse run store (ADR 0029, a prototype)

With `QUERYAPIGATE_RUN_STORE_URL` set, run history, the event log and usage rollups are kept in ClickHouse rather than the metadata store (ADR 0029). Keys, queries, connections and the audit log stay in the metadata store, so the run store never decides who may do what. Where it is used:

- **What its credentials reach.** The run store holds every run's history entry - who called, which query, when, how many rows, the error message, an ad-hoc statement as `QUERYAPIGATE_HISTORY_ADHOC_SQL` keeps it - but never a result row. A stolen credential reads those entries, and one that can insert can forge runs and events: history and live events would show calls that didn't happen. It can't change a key, a query or a grant. Give the data plane a user that can only insert into `events` (guide 65); its rows reach history and the rollups through views that run as the user who made them. What such a writer puts in a row stays inside its own event: a frame's data is always one line (a line break can't start a frame of the writer's own), and ids above 2^53 are refused by the table.
- **In transit:** `https://` is required except to a loopback host, since the URL carries a password and every run. It is a secret setting: it can be read from a file (`QUERYAPIGATE_RUN_STORE_URL_FILE`), is masked in Settings, and only its host, port and database are written to the metadata store or the log.
- **Queries are parameterised.** Every value a read compares against goes to ClickHouse as a typed query parameter (`{key:String}`), never in the SQL text; table names are the module's own constants.
- **Where runs are, recorded once.** The metadata store records which ClickHouse server and database hold its runs. An instance configured otherwise refuses to start, so a misconfigured instance can't send a share of the runs elsewhere or leave them out of an erasure. A data plane learns the location from its lease, and writes nothing while its own differs.
- **Erasure (ADR 0016)** reaches `events`, `history`, `history_counts` and both rollup tables, which hold key names. Rows are hidden at once by a lightweight delete and gone from disk once ClickHouse has rewritten the parts that held them (`APPLY DELETED MASK`). It deletes again once runs still in flight - a data plane's batch, another worker's

queue - would have landed. Ad-hoc statements, which may hold values, are kept as text no longer than the event log: the hourly rollup, kept 400 days, holds their hash. A backup of the metadata store no longer holds runs; ClickHouse's own backups do, and an erasure doesn't reach those.

- **Event ids** come from blocks of each millisecond that writers lease in the metadata store, so two writers never mint the same one; the data plane leases its block from Python, behind its secret, keeping its own store access read-only. Delivery waits a settle window so a resume misses nothing; an event acknowledged later than that is counted (`queryapigate_events_late_total`).

Reviewed on 2026-10-28, part of what it needs to graduate (ADR 0017): six findings, fixed in schema version 2 of its tables (ADR 0029 section 7). It stays experimental until a release unchanged (the 250-million-run measurement is done).

Secrets at rest

Connection passwords in the store can be a literal string, an `"${ENV_VAR}"` reference resolved from the process environment at load time (never written back to disk), or - with `QUERYAPIGATE_SECRET_KEY` set - Fernet-encrypted at rest. Encryption is **opt-in, not the default**; without a secret key, a connection password is only as protected as the file/volume it sits in. The key itself must be kept outside `QUERYAPIGATE_HOME` and out of version control - there is no recovery path for an encrypted password without it, by design, the same property that makes it real encryption rather than obfuscation. API responses always mask stored passwords regardless of how they're stored. The key can be rotated without taking a connection down (ADR 0009): `QUERYAPIGATE_SECRET_KEY` lists the new key first and the old one after it, `queryapigate rekey` re-encrypts every stored password under the new one in a single transaction (all or nothing), and the old key is then removed. A key suspected leaked is replaced the same way; backups taken before the rekey still open with the old key, so they need retiring with it.

The audit trail

The audit log is a hash chain (ADR 0012): each entry's `hash` covers the previous entry's, so changing, removing, inserting or reordering one breaks every hash after it, and `queryapigate audit verify` (and a check every 5 minutes) names the first. Against someone who can write to the store, a plain hash is only consistency - they can recompute it - so with `QUERYAPIGATE_AUDIT_KEY` set the chain is HMAC-signed with a key kept outside the store, and a key once set means every newer entry must carry it. What the chain can't show alone: the newest entries deleted (its end can't know what came after it), and a rewrite by someone holding both the store and the key. Both are why a copy outside - the SIEM stream, or the export file shipped elsewhere - matters. The stream sends each entry with its `seq` and `hash` over TLS - the server's certificate verified, the client's optional - so a later rewrite of the store contradicts what the SIEM already has; its delivery is at least once, and syslog over TCP can lose the batch sent to a SIEM that dies without closing the connection (OTLP, acknowledged per batch, doesn't). `queryapigate audit verify --file` checks the SIEM's copy on its own - repeats from at-least-once delivery dropped only when exact.

Reviewed for 0.32 (ADR 0019). Looked at, against rsyslog, syslog-ng and the OpenTelemetry Collector rather than a receiver of our own: the server's certificate verified against the configured CA; a missing client certificate, and one from another CA, refused by the receiver and noticed before anything is counted delivered; a receiver stopped with a connection open, noticed rather than written into; the cursor and lease surviving it; nothing in a message but the entry as the store holds it - no parameter values, which entries never carry. **Found:** a SIEM's copy of an untouched log failed `audit verify --file` as soon as at-least-once delivery had repeated one entry; fixed - exact repeats are dropped and counted, a repeat that differs is `conflicting_copies`.

The key's guarantee starts at the `audit_key_set` entry the server writes when it first starts with a key. Someone with write access to the store, while the server is stopped, could delete the keyed history and leave an unkeyed one: at the next start the server sees an unkeyed chain, as when a key is first set, and writes `audit_key_set` again. It records how

many entries came before it unkeyed (`entries_before_unkeyed`) - a second `audit_key_set`, or one counting entries you know were keyed, is the sign - but only a copy outside the store proves which history is real: the SIEM's already holds the keyed hashes.

Transport

QueryAPIGate does not terminate TLS itself - that is explicitly the reverse proxy's job (Caddy, nginx, Traefik; see DEPLOYMENT.md). An API key travels in a request header, so serving over plain HTTP means the key is recoverable by anyone who can observe the traffic - TLS termination in front of QueryAPIGate is not optional once a key exists.

Response headers don't depend on that proxy (ADR 0010): every response sets `nosniff`, `Cache-Control: no-store` (a cached saved query `private`, never shareable: callers authenticate with `X-API-Key`, which HTTP's shared-cache rule doesn't cover - the 0.22.1 fix) and, on API responses, a CSP that runs nothing and a `no-referrer` policy. HSTS is the deployer's choice (`QUERYAPIGATE_HSTS_MAX_AGE`), sent only over HTTPS. `/docs`, where people type an API key, loads one exact Swagger UI release with Subresource Integrity hashes and a CSP allowing only those files: a compromised CDN, or a bad release of the library, can't run script there. The Console bundles everything it runs.

Tracing (ADR 0010, experimental, off unless an OTLP endpoint is set) sends spans to a collector the deployer runs. Its readers are treated like the audit log's: no data access. So spans carry the route, the saved query's name and the row count - never a parameter value, row, statement, header or credential, and not exception messages, which can carry values; a test enforces it on every front door. A caller's `traceparent` is accepted, which also lets it ask for its trace to be sampled: with the SDK's default sampler, a caller can raise how much is exported. Deployers who care set `OTEL_TRACES_SAMPLER=traceidratio`. The OTLP endpoint's headers (`OTEL_EXPORTER_OTLP_HEADERS`, often a token) are never shown in Settings. Spans do carry two things that can be personal data, as run history does: the caller's address (`client.address`) and who called (`queryapigate.caller` - for a signed-in user `jwt:<subject>`, which may be an email address) - keep the collector's retention to what that allows. An MCP client's tool name is recorded only if it has the shape a saved query's name can have; anything else is `(invalid)`.

On Kubernetes (the Helm chart, ADR 0013)

The chart's pods run as UID 1000, with a read-only root filesystem, every capability dropped, the RuntimeDefault seccomp profile, and no service account token: QueryAPIGate never calls the Kubernetes API, so a compromised pod has no credential for it. The server writes only to `/data` (the store's volume, or an empty directory on PostgreSQL) and `/tmp`. What stays the deployer's:

- **Secrets in a Kubernetes Secret are only as safe as the cluster.** Anyone who can `get secrets` in the namespace, or read etcd unencrypted, reads the API key, the Fernet key and the database URL. Keep that RBAC narrow, and encrypt Secrets at rest. With `secret.create`, the values are also in your values file and in Helm's release record (itself a Secret in the namespace). `existingSecret` keeps them out of both.
- **The chart refuses a secret setting in `config`**: the keys, the database and Redis URLs, the SSO client secret and OTLP headers. `config` is plain text in the Deployment, where `get deployments` reads it.
- **NetworkPolicy is off by default**, because the right rules depend on the cluster. Without it, any pod in the cluster can reach the API and the MCP server. `networkPolicy.enabled` limits who can connect in, and narrows what a pod can reach out to, to `egress` - so a stolen admin key can't point a new connection at an internal address it doesn't list.
- **The chart and image are signed by this repository's release workflow** (from 0.31, ADR 0018): keyless, by digest, the certificate naming `docker.yml` at the release's tag, the signature in Sigstore's public log - with SLSA provenance, an SBOM and a GitHub attestation naming the commit. So a mirror, a moved tag or a look-alike repository can't pass for a release: verify before you run it, and have the cluster refuse anything else (guide 59).

> What a signature doesn't say: that the code is right. Someone who can push a tag here can still sign a release - the log then records it. What it's built on is pinned too: base images by digest, every action by commit. Releases before 0.31 aren't signed.

Reviewed for 0.30 (ADR 0017). Looked at: every pod's security context, the service account token left unmounted, `vaultToken` a projected token for Vault alone, the secret settings refused in `config`, the routes, what the NetworkPolicy allows. **Found:** the ingress routes every path to the API server, `/metrics` included - and `/metrics` was open, with API keys', queries' and connections' names in its labels, and each administrator's own name (often their username, for one signed in through SSO). So an install with the ingress on published them. Now `metricsAuth` (on by default) gives `/metrics` a token from a Secret of the chart's own, kept across upgrades, and the ServiceMonitor sends it; and administrators are counted under one `admin` label, as signed-in users are under `jwt`.

Configuration as code (ADR 0014)

A configuration file can change anything an owner can - connections, grants, keys - so applying one is an owner's capability (`config.apply`), and so is reading what applying one would do. What keeps a file safe to put in git, and what's still the deployer's:

- **A file never holds a secret.** A password must be a `${VAR}` reference - a literal is refused - and a reference to the server's own settings or credentials, or to a variable `QUERYAPIGATE_SECRET_ENV_VARS` doesn't list, is refused, as through the API (a connection pointed at a host of the author's would otherwise send it there). Export turns stored values into references. Not checked: a secret inside another field - a JDBC URL with `password=` in it - which is in the file as it is in the store.
- **A key is in a file by its secret's hash.** That's safe to publish only because the secret is 32 random bytes: `config new-key` makes one. A key someone gave a guessable secret could be found from its hash, offline. Two keys can't share a secret.
- **Who can change the file can change the server.** With `QUERYAPIGATE_CONFIG_FILE` (the chart's `configAsCode`), anyone who can edit the file - the repository's main branch, the ConfigMap - has an owner's reach at the next start. Protect the branch and require review; that is the control this feature exists to give.
- **Applying is all or nothing, and audited** - each change `via: config`, under one `config_applied` entry with the file's hash and who applied it. A rolled-back apply leaves no audit entry, in the store or the export file.
- **`QUERYAPIGATE_CONFIG_LOCKED`** keeps what a file manages to the file for the API and the Console. It doesn't bind someone with a shell on the server: the CLI works on the store directly, as it always has.
- **Pruning deletes only what a file manages**, and what the API deletes is no longer managed - so a later object of the same name, made in the Console, isn't pruned as the file's.
- **A file is capped at a million values**, YAML aliases counted as they expand, so a small file can't be a huge one.

Reviewed for 0.30 (ADR 0017). Looked at: who may diff and apply (`config.apply`, owners); every object checked by the function the API uses for it, so a file can't do what the API refuses; the `--server` CLI, now tested against a real `queryapigate serve` - its key only from `QAG_API_KEY`, no store opened locally; startup apply under the store's write lock, once per file; pruning only what a file manages; the lock's reach. **Found:** a `${VAR}` reference could name the server's own cloud credentials - `${AWS_SECRET_ACCESS_KEY}` with a host of the author's - through a file as through the API. Fixed in both (Query execution, above): those credentials are always refused, and `QUERYAPIGATE_SECRET_ENV_VARS` limits the rest to what the deployer lists.

Secrets from secrets managers (ADR 0015)

A reference - `${file:...}`, `${vault:...}`, `${aws-sm:...}` - or a connection's `credentials` lets an administrator make the server fetch a secret and use it as a password. The server's identity (its files, Vault token, IAM role) reaches far more than any one connection should, and an administrator could point a connection at a host of theirs to be sent whatever it fetches. So:

- **Nothing is reachable until the deployer says so**: `QUERYAPIGATE_SECRET_FILE_DIRS`, `QUERYAPIGATE_VAULT_PATHS`, `QUERYAPIGATE_VAULT_DATABASE_ROLES`, `QUERYAPIGATE_AWS_SECRET_PREFIXES` and `QUERYAPIGATE_AWS_RDS_HOSTS`, checked when a reference is saved and again when it's used. The store's own folder is never readable, and the server's own `QUERYAPIGATE_*` / `OTEL_*` settings never reachable, by any reference - nor, through `${file:...}`, a file one of them or the server's own credentials name, the Kubernetes and EKS service account tokens, or `/proc`. A file is opened at the real path that was checked, so a symbolic link changed in between can't point it elsewhere.
- **A Vault database role can be bound to the host its credentials may go to** (`reporting-ro@warehouse.internal`), and an RDS IAM token is made only for the listed hosts. **Bind every role**: an unbound one hands its credentials to whatever host a connection names - including through `POST /api/v1/connections/test`, without saving anything.
- **What an allowed reference names, an administrator can still send elsewhere** - a `${vault:...}` password to a host of theirs - exactly as with `${VAR}`: an administrator is trusted with the credentials of the connections they manage (as above). The allow-lists bound which; short-lived credentials bound for how long; a bound role, where.
- **Resolved values are held in memory only**, for at most their lease or `QUERYAPIGATE_SECRET_CACHE_SECONDS`, and never written to the store, a log, a response, a trace or the audit log; errors name the provider and its reason, never a value. A provider that fails is forgiven only within what it gave before; past that, `secret_unavailable`.
- **IAM tokens and Vault credentials are passwords**: `aws-rds-iam` is refused without TLS, and a PostgreSQL or MySQL connection's TLS settings now reach its driver (before 0.28 they were dropped).
- **The Vault token, AppRole secret ID and the chart's projected token** are the server's to protect like `QUERYAPIGATE_SECRET_KEY`. Vault's policy for that identity is the outer limit of what any reference can reach - keep it to the paths and roles connections need. The same goes for the IAM role.
- **Leases are revoked when a process stops** (on SIGTERM too), so a stopped instance's database users don't linger; one that crashes leaves them to their TTL.
- **Why a secret couldn't be had is the operator's, not the caller's**: `secret_unavailable` says only that, and the reference and the provider's error go to the log and to `detail`, which a scoped key calling a saved query doesn't get - they name where the server's secrets live, and its Vault's address.

Reviewed for 0.34 (ADR 0021) - `${VAR}`, `${file:...}`, `${vault:...}` and `vault-database`, against Vault 1.21. Looked at: each allow-list checked when saved and again when used, and Vault paths and roles matched whole (a `reporting` prefix doesn't reach `reporting-admin`); a file opened at the path that was checked; the cache failing closed, and a failing provider asked at most every 10 seconds; the Vault token's sign-in and renewal, a forbidden answer signing in again only once; what an error tells whom. **Found, and fixed:** a `${file:...}` could read the server's own secrets wherever they were files inside an allowed folder - `QUERYAPIGATE_SECRET_KEY_FILE=/run/secrets/fernet` beside `QUERYAPIGATE_SECRET_FILE_DIRS=/run/secrets`, as the docs suggest - and so have the server's key sent to an administrator's host as a password; and, with a broad folder, `/proc/self/environ` or the service account's token. And `secret_unavailable` told any caller the reference and Vault's own answer. AWS (`${aws-sm:...}`, `aws-rds-iam`) stays experimental, tested only against a stand-in.

What is kept, and the cryptography it rests on (ADR 0016)

Where data goes is the inventory; what bears on threats:

- **Values stay out of what's kept by default.** Run history keeps parameter names, never values; traces never hold one; the access log has paths without query strings and no referrer (`QUERYAPIGATE_ACCESS_LOG_QUERY` puts the query string back - then the log platform holds whatever callers put in URLs). What can still carry a value: an ad-hoc statement's text in history and the log (`QUERYAPIGATE_HISTORY_ADHOC_SQL=hash` keeps both to its SHA-256), a database's own error message in the log, an agent's write arguments for 30 days, an export's watermark, and the response cache's rows. Treat the log platform and the store as holding personal data.
- **Erasure is an owner's, and it isn't silent.** `history forget` deletes a caller's runs and usage counts - any caller, a key's too, so an owner could clear the record of what a key ran. The audit log records each erasure, its counts and the caller's SHA-256 (key names are few: the hash names the key to anyone who has the list), so it can be seen, not undone. The audit log itself can't be erased from without breaking its chain (ADR 0012).
- **The cryptography is FIPS-approved algorithms only**, enforced by a test over the code; whether it runs in a validated module is the platform's (a FIPS-mode OpenSSL, the distribution's `cryptography`). The page names what is outside that boundary - the PostgreSQL and MySQL drivers' binary wheels, DuckDB's TLS, a JDBC or H2 connection's Java runtime - and CI runs the whole suite under OpenSSL's FIPS provider.

Live events (ADR 0020)

`GET /events` - on the main server, and `queryapigate events` for many clients - streams each run as it happens: which query, on which connection, by which key, how it went, and an ad-hoc run's SQL as history keeps it. So:

- **Never public.** A real key or a signed-in user's token, in a header - never the URL, where it would reach access logs - unless the server runs with no keys at all. A stream is checked again every minute, so a revoked key or an expired token loses it within about a minute and a quarter.
- **The admin key and data administrators see every run; anyone else only their own** - the runs made with that key, or by that user, since the key was created. Filters only narrow that, and their values are bound, never SQL. Key names can't hold the `:` of `jwt:<subject>`, and an administrator's name can't be a key's.
- **Resource use is bounded**: `QUERYAPIGATE_EVENTS_MAX_CONNECTIONS` streams a process, `QUERYAPIGATE_EVENTS_MAX_STREAMS` on the main server (each a request thread), `QUERYAPIGATE_RATE_LIMIT` before a key is even checked, 10 seconds and 100 headers for a request, and a client that stops reading cut off once 1,000 events wait for it. Not bounded: how many of those streams one key holds.
- **Kept for a while, and forgettable**: the event log keeps `QUERYAPIGATE_EVENTS_RETENTION_HOURS` (24 by default), and `history forget` erases a caller's events with their runs.

Reviewed for 0.34 (ADR 0021). Looked at: authentication on both endpoints and the recheck; who sees what, live and on a resume, for keys, signed-in users and each administrator role; the filters' SQL; the request parser's limits; CORS and the security headers; `/metrics` behind its token. **Found, and fixed:** a deleted key's name can be given to a new key, and the event log kept the old key's runs under it - the new key could resume from 0 and read its predecessor's runs of the last day. A key now sees no event from before it was created.

Rate limiting and abuse prevention

The rate limiter throttles both by source IP (server-wide) and, additively, by API key (`rate_limit` grant). Without `QUERYAPIGATE_REDIS_URL` its state is in each process's memory, so running more than one process silently multiplies the effective limit. With it, every instance and process counts against one budget in Redis (an atomic token bucket, on Redis's own clock); if Redis becomes unreachable, each process falls back to its own counting - limits stay enforced, per process, never lifted - and the `rate_limits_not_shared` alert and a metric say so. `QUERYAPIGATE_TRUST_PROXY` tells the server how many reverse-proxy hops to trust when resolving the real client IP

from `X-Forwarded-For` - left at its default of `0` with a direct-connecting client, or set to the actual hop count behind a proxy; getting this wrong in either direction lets a client forge its apparent IP and evade both the rate limiter and `allowed_ips` checks.

A rate limit counts calls, not what they cost, so a key under its rate can still hold every pooled connection with slow queries and starve everyone sharing them. A key's or role's own `max_concurrent`, `query_timeout` and `max_rows` (ADR 0011) bound that: queries running at once (a stream counts until its last row), how long each may run, how many rows a call returns - each only ever narrowing the server's limits. Counted in Redis across instances, with leases so a crashed process can't hold a caller's slots forever; without Redis, per process, as rate limits are. Usage quotas cap a caller's calls and rows per day or month. They're a contract's meter, not a hard security boundary: a caller overshoots by its calls in flight, and with several instances but no Redis by up to a quota per instance in a short burst. `max_concurrent` and the rate limit are what bound a burst.

Reviewed for 0.30 (ADR 0017). Looked at: who sets a quota (`access.write`, never the caller; a configuration file's role through the same validation); that it's checked before the response cache, so a cached answer counts and is refused when used up; that MCP tool calls and REST calls share one counter per caller, and per user for a role's signed-in callers; the Redis keys (`qag:quota:<caller>:<period>`), where a period always ends in its date, so two callers' keys can't meet whatever their names; that `forget` matches Redis keys with the caller's glob characters escaped; who reads every caller's counts (`GET /api/v1/usage/quotas`, `observe` - callers' names, personal data for a signed-in person, as in run history). Found nothing to fix; the documentation now says what doesn't count (exports, live events, MCP resources, the Management API) and that an agent's write request does.

CORS

Cross-origin access is off unless `QUERYAPIGATE_CORS_ORIGINS` is explicitly set, and CORS is a browser-only control, not authentication - it stops a browser tab from making credentialed requests on a site's behalf, it does nothing against a direct HTTP client. Setting it to `*` without also requiring an API key means any site's script can call the API directly from a visitor's browser.

Audit logging and data at rest

Every configuration change (`record_audit(actor, action, target, changes)`) is appended to an `audit_log` table - the actor (the administrator's name, or `admin` for the shared key), how they authenticated (`via`: their own token, the break-glass key, the CLI), the action, the target, and a JSON diff of what changed. Only administrators can read it, and nothing in the API can alter it; a scoped key cannot see it. A separate `execution_history` table records individual query runs (caller, row count, timing), independent of the audit log's configuration-change trail. **The audit log is tamper-evident** - a hash chain, signed with `QUERYAPIGATE_AUDIT_KEY` (The audit trail); keep a copy outside the store too (the SIEM stream, or `QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE` shipped elsewhere), since only that catches its newest entries deleted. **Run history is not**: a plain table, which someone with direct database access (not API access - the API never exposes a way to alter history) could edit without detection.

Data at rest: what one file actually holds

Everything lives in a single SQLite file, `queryapigate.db`, inside `QUERYAPIGATE_HOME` - connections (with passwords plaintext, `${VAR}`-referenced, or Fernet-encrypted per the setting above), saved queries and every version's full SQL text, execution history (including row counts), API key *hashes* (never secrets) and grants, roles, administrators and their admin tokens' *hashes*, and the audit log. Protecting this one file - filesystem permissions, backup encryption, who can reach the volume - is entirely the operator's responsibility; `QUERYAPIGATE_SECRET_KEY` only ever covers connection passwords, not the rest of what the database holds.

MCP server

`queryapigate mcp` (a separate process from `queryapigate serve`) reuses the identical authentication and authorization path - the same `X-API-Key` header, the same `apikeys.authenticate()`, the same per-key `connections` / `queries` / `collections` scoping `GET /catalog` already enforces, the same open-access fallback when no server key is configured. A saved query that writes is a tool only when its author marked it for agents and the caller's grant allows the write - and then each write is approved by a person first, unless it only inserts (below); otherwise it isn't listed, and isn't run when called by name or by version (fixed in 0.20.1). `execute_sql` never writes. One transport-specific note: an MCP client has to be configured to send the key as a request header (Streamable HTTP clients can) - never as a URL query parameter, which would leak it into access logs. The MCP server doesn't read `QUERYAPIGATE_TRUST_PROXY`: behind a proxy it sees the proxy's address for every caller, so `allowed_ips` and the server-wide rate limit can't tell MCP callers apart there.

Every request to `/mcp` is authenticated before the MCP session sees it, and refused at the HTTP level (`401`, `403`, `429`) - nothing of the protocol is reachable without a credential once one is required.

OAuth for AI clients (ADR 0006)

With `QUERYAPIGATE_MCP_RESOURCE_URL` and `QUERYAPIGATE_MCP_AUTH_SERVER` set, the MCP server is an OAuth 2.1 resource server: it checks tokens; the organisation's identity provider issues them. What that protects, and what it doesn't:

- **Tokens are bound to the MCP server.** A token's `aud` must be the MCP server's own URL, its `iss` the configured provider, its signature one of the provider's published keys with that key's own algorithm (never `none`, never an HMAC "signed" with a public key), its `exp` in the future, and it must carry `QUERYAPIGATE_MCP_SCOPE`. A token issued for any other application of the same provider - including this server's own REST API - is refused, and REST refuses a token naming the MCP server; startup refuses a REST audience that includes the MCP URL. A leaked token works against one service only, until it expires.
- **No token is passed on.** Databases are reached with each connection's own credentials; a caller's token never leaves the MCP server.
- **The provider's keys are found through its metadata**, fetched over `https://` only (a redirect to plain HTTP is refused), from documents that must name the configured issuer as their own (RFC 8414) - a metadata document claiming to be another issuer isn't used. A provider that can't be reached means every token is refused, never accepted unchecked.
- **One person, one name.** An agent's runs are under `jwt:<sub>` - the name that person has signed in to REST - when REST trusts the same provider; under `mcp:<sub>` when REST trusts another, so that two people whose `sub`s happen to match never share a name, rate limit, history or event stream.
- **What remains with the provider:** who may get a token for the MCP server, with which scope and which role claim, is the provider's configuration - QueryAPIGate trusts what a valid token says. Review that the role claim can't be set by users themselves (a self-editable profile attribute mapped into the token would let anyone pick a role), and how clients register: anonymous Dynamic Client Registration lets anyone register a client in your realm.
- **Revocation** is the provider's too: a token is valid until it expires. Keep access tokens short-lived; disabling or narrowing the role in QueryAPIGate takes effect on the next request.

Reviewed for 0.30 (ADR 0017). Looked at: the algorithm taken from the provider's published key, never the token's header, and only an asymmetric one; `iss`, `aud` and `exp` required; tokens for REST and for the MCP server kept apart in both directions; the metadata's issuer and `https://` endpoints, and no redirect off `https://`; one name per

person across two issuers (`mcp:<sub>`); the role claim naming only roles that exist. The signing keys come through PyJWT's own client, which follows no redirect at all - from 2.14, the floor CI tests; a test now holds it to that. Found nothing to fix.

Agents that write (ADR 0008)

- **Only what an author marked, only data changes.** A write tool is a saved query marked `agent_writes` - one `INSERT` , `UPDATE` or `DELETE` , never DDL, checked when saved - for a caller whose per-query grant allows the write, with `QUERYAPIGATE_ALLOW_WRITES` on. The grant, the mark and the server's setting are all checked again on every call, so taking any of them back stops the next one, approved or not.
- **A named person approves each write, and that write only.** The approval is bound to the caller, the query's version and the exact arguments (by hash), runs once, and expires. A model can't satisfy it by itself: the credential that asked can never approve (`own_write`), so an MCP client holding an approver's token or the shared key can't approve its own writes - give agents their own keys.
- **A ceiling on what any write changes.** It runs in a transaction, rolled back when it changed more than its `max_affected_rows` - counted when it runs, not estimated when approved. On MySQL that needs transactional (InnoDB) tables.
- **Insert-only queries skip the approval**, and can add rows but never change or remove one - choose that mode only where an added row can do no harm on its own.
- **What remains:** an approver judges what they're shown - the SQL and the arguments, not which rows; and a description, a row or a document the agent read can still talk it into *asking* for a write. The approval is where that stops.
- **What's recorded:** every request 30 days, with who decided and how they signed in; the run in history with its approver; the audit log with the arguments' names and hash, never their values, since auditors read it.

Reviewed for 0.30 (ADR 0017). Looked at: an approval bound to the caller, the query's version and the arguments' hash; approved to running in one conditional update, so it runs once however many calls race; deciding guarded the same way; the credential that asked never the one that approves; the grant, the mark and the server's setting checked again on the call that runs it, before the approval is claimed; at most 100 pending per caller; insert-only's idempotency key scoped to the caller and the tool. Found nothing to fix.

What agents are told, and what they did

- **Descriptions are instructions-shaped data.** A table's or column's description, and a connection's notes, are read by agents as context (descriptions). Whoever can write them can steer what an agent does with the data it can reach - not what it can reach. Writing them takes `connections.write` (owners and admins), and every change is audited. **Importing from the database's comments** brings in text written by whoever has `COMMENT` rights in the database: review an import (it says `source: "database"` on each) before relying on it.
- **Descriptions reach no further than the schema.** A caller sees those of the tables it can reach, and a caller restricted to some tables never gets the connection's notes, which may name any table.
- **Agent usage** (`GET /api/v1/usage/agents`) shows administrators each agent's ad-hoc SQL, as run history does - it is readable by every administrator role, auditors included. With `QUERYAPIGATE_HISTORY_ADHOC_SQL` set to `hash` or `none` , it shows hashes or nothing.

Non-goals: what this does not defend against

Being explicit about these is as important as everything above:

- **QueryAPIGate is not a network perimeter or a web application firewall.** It assumes it is reachable only by callers who should be attempting requests at all - a reverse proxy, network policy, or VPN is still the operator's job for anything beyond that.
- **An owner, admin or developer is equivalent to database-superuser-scoped-to-your-connections** for data. Roles limit what each can *change* in QueryAPIGate, not which rows they can read; database-level least-privilege grants on the connection account itself are the actual backstop here, not a QueryAPIGate setting.
- **The regex-based read-only/single-statement guard is defence in depth, never a substitute for least-privileged database credentials.** Connect with an account that only has the grants the API actually needs; assume the guard could someday have another gap like the one already found and fixed.
- **Several instances are one service only on a shared store and Redis.** With a PostgreSQL store and `QUERYAPIGATE_REDIS_URL`, every instance shares keys, connections, the audit trail and rate limits; without Redis, each counts limits on its own (the `instances_not_shared` alert says so), and two replicas each with their *own* SQLite volume are two separate servers with their own keys, connections and audit trail. `/metrics` is always per process - scrape each one.
- **Run history is not tamper-evident, and the audit log's chain has limits.** The chain shows an edit, but not its newest entries deleted, nor a rewrite by someone holding both the store and `QUERYAPIGATE_AUDIT_KEY` - a copy outside the store (the SIEM stream) is what proves those (The audit trail).
- **Connection metadata (hosts, ports, usernames, database names) is visible to every administrator role** - `GET /api/v1/connections` is administrators-only, and passwords are always masked, but the rest is not treated as secret.
- **Names can be probed.** A saved query a key can't reach answers `403` naming its connection; a name that doesn't exist answers `404`. Don't put anything sensitive in query or connection names.
- **An agent may be steered by what it reads.** Data in results, descriptions and notes all become part of an AI agent's context. QueryAPIGate limits what an agent can reach and hides what masking covers; it can't make an agent ignore instructions planted in data it is allowed to read.
- **`/metrics` needs no API key** - a scraper shouldn't hold one - **and is open until `QUERYAPIGATE_METRICS_TOKEN` is set**: then only a request with that bearer token, or an administrator who may observe, reads it. Its labels name API keys, queries and connections (administrators and signed-in users only as `admin` and `jwt`): set the token wherever the server can be reached from outside. The Helm chart does.

Where to go from here

- Where data goes - what is kept, where, for how long and who reads it; the record of processing this page's controls protect.
- SECURITY.md - the operational hardening checklist this page explains the reasoning behind, and where to report a real vulnerability.
- DEPLOYMENT.md - TLS termination, secrets in an env file, backups, and the single-worker rationale in a real production Docker setup.
- API.md - the full, field-by-field reference for every grant on a scoped API key.